Enseignante : Dorra DHAOU
dorradhaou.isimm@gmail.com

TP N°2 – SUITE -

Matière : Programmation Mobile

Niveau : ING1 Info, L3 Info, L3 Tic

AU : 2026-2027

L'objectif de cette suite de TP est de :

- Utiliser correctement les **ressources textuelles (strings.xml)**, ce qui facilite la **traduction** de l'application (internationalisation), la **réutilisation** et la **maintenance** du code, et le **respect des bonnes pratiques Android**.
- Apprendre à **transmettre des données entre activités** en utilisant les **Intent** et les **Extras** (putExtra() / getXExtra()).
- Maîtriser la communication **retour** entre activités Android, utiliser les constantes RESULT_OK et RESULT_CANCELED et Manipuler les Intent dans les deux sens (aller-retour).

Partie 1 : Amélioration de l'application

Avant de passer à la transmission de données entre les activités, nous allons améliorer la structure de notre application pour la rendre plus propre et plus facilement maintenable.

Dans cette partie, l'objectif est de **séparer les chaînes de caractères du code source**. Pour cela :

1. **Ouvrez le fichier res/values/strings.xml.**
    - Définissez-y toutes les chaînes de caractères utilisées dans les activités CurrencyActivity et ComputeActivity (boutons, labels, messages, toasts, titres, etc.) comme suit :

```
<!-- ComputeActivity  -->
    <!-- Hints et labels -->
    <string name="radio_sum">Somme</string>
    <string name="radio_square">Carré</string>
    <string name="hint_value1">Valeur 1</string>
    <string name="hint_value2">Valeur 2</string>
    <string name="btn_calculate">Calculer</string>
    <string name="tv_result">Résultat</string>
    <!-- Toast messages -->
    <string name="msg_enter_two_integers">Veuillez saisir 2 entiers valides</string>
    <string name="msg_enter_one_integer">Veuillez saisir un entier valide</string>
    <string name="msg_select_operation">Sélectionnez une opération</string>
    <!-- Formats -->
    <string name="format_result">Résultat : %1$d</string>
```

```
<!-- CurrencyActivity -->
<!-- Hints et labels -->
<string name="hint_amount">Donner un montant...</string>
<string name="btn_dt_to_euro">DINAR TUNISIEN → EURO</string>
<string name="btn_euro_to_dt">EURO → DINAR TUNISIEN</string>
<string name="txt_result">Résultat de Conversion</string>
<!-- Toast messages -->
<string name="msg_enter_amount">Veuillez entrer un montant</string>
<!-- Formats -->
<string name="format_dt_to_euro">%1$.2f DT = %2$.2f €</string>
<string name="format_euro_to_dt">%1$.2f € = %2$.2f DT</string>
```

2. **Modifiez les fichiers XML d'interface** (activity_currency.xml, activity_compute.xml)

- Remplacez les textes en dur par les références aux ressources définies. Par exemple :

```
android:text="@string/btn_calculate"
android:hint="@string/hint_value1"
```

3. **Adaptez le code Kotlin si nécessaire.**

Par exemple, pour afficher un message avec un texte défini dans `strings.xml` :

```
Toast.makeText(this, getString(R.string. msg_enter_one_integer), Toast.LENGTH_SHORT).show()
```

4. **Ouvrez le fichier res/values/colors.xml** et définissez les couleurs utilisés dans votre application. Par exemple les couleurs spécifiques aux boutons active et inactive dans CurrencyActivity:

```
<!-- Couleurs spécifiques aux boutons -->
<color name="btn_active">#1565C0</color>          <!-- Bouton actif -->
<color name="btn_inactive">#B0BEC5</color>        <!-- Bouton inactif -->
```

btn_active et **btn_inactive** : seront utilisées dans le code Kotlin à la place de **android.R.color.holo_blue_dark** et **android.R.color.darker_gray**.

5. **Adapter le code Kotlin pour utiliser les couleurs définies :** Par exemple :

```
binding.btnDtToEuro.setBackgroundColor(getColor(R.color.btn_active))
```

Enseignante : Dorra DHAOU
dorradhaou.isimm@gmail.com

Partie 2 : Transmission de données entre activités

Dans cette partie, nous allons **modifier le comportement de l'application** afin que l'activité **CurrencyActivity** ne réalise **plus directement les conversions**.

Désormais, les calculs seront effectués dans **une nouvelle activité** nommée **ConversionActivity**.

Pour cela :

1. **Créer une nouvelle activité**

   Ajoutez une activité de type **Empty Views Activity** appelée **ConversionActivity**.

2. **Modifier CurrencyActivity**

   - Lorsque l'utilisateur clique sur l'un des deux boutons ("DT → €" ou "€ → DT"), on **crée un Intent** pour lancer ConversionActivity.
   - On y **transmet** :
     - Le **montant saisi** par l'utilisateur.
     - Le **type de conversion** (par exemple "DT_TO_EURO" ou "EURO_TO_DT").

NB : Utilisez les méthodes putExtra() pour le chargement de l'Intent avec les données à transférer et get*X*Extra() pour leur récupération.

Exemple :

```
val intent = Intent(this, ConversionActivity::class.java)
intent.putExtra("amount", edtAmount.text.toString().toFloat())
intent.putExtra("conversionType", "DT_TO_EURO")
startActivity(intent)
```

Explication :

La méthode **putExtra()** permet de **joindre des données** (comme un montant ou un texte) à un **Intent** avant de démarrer une autre activité.

Exemple : intent.putExtra("clé", valeur)

3. **Dans ConversionActivity**, récupérez les données envoyées :

```
val amount = intent.getFloatExtra("amount", 0f)
val type = intent.getStringExtra("conversionType")
```

Explication :

La méthode **getXExtra()** (où X représente le type de donnée, comme *Int*, *Float*, *String*, etc.) permet à l'activité réceptrice de **récupérer ces données**.

Exemple : val montant = intent.getFloatExtra("clé", valeurParDéfaut)

4. **Effectuez la conversion et affichez le résultat** : Selon le type de conversion :

```
val rate = 3.4
val resultText = when (type) {
  "DT_TO_EURO" -> "L'équivalent de $amount DT en Euro est : %.2f Euro".format(amount / rate)
  "EURO_TO_DT" -> "L'équivalent de $amount Euro en DT est : %.2f DT".format(amount * rate)
    else -> "Type de conversion inconnu"
}
tvResult.text = resultText
```

Résultat attendu

Lorsqu'un utilisateur saisit 100 et clique sur :

"DT → €", l'application affiche :

➜ « L'équivalent de 100 DT en Euro est : 29.41 Euro »

"€ → DT", l'application affiche :
➜ « L'équivalent de 100 Euro en DT est : 340 DT »

5. **Améliorer le code précédent** en définissant les chaînes de caractères utilisées dans le fichier **strings.xml** afin de les séparer du code comme suit :

```
<string name="conversion_result_dt_to_euro">L'équivalent de %1$.2f DT en Euro est :
%2$.2f €</string>
<string name="conversion_result_euro_to_dt">L'équivalent de %1$.2f € en DT est :
%2$.2f DT</string>
```

Partie 3 : Retour de résultats

Dans cette partie, nous allons modifier le comportement de l'application pour permettre à l'activité ConversionActivity de **renvoyer le résultat de conversion** vers l'activité principale CurrencyActivity. Ainsi, l'activité principale CurrencyActivity récupère le résultat calculé dans **ConversionActivity** et pourra **afficher le résultat final** dans son TextView, ou bien **indiquer qu'aucun résultat n'a été renvoyé**.

Pour cela :

1. **Ajoutez deux boutons dans ConversionActivity**
    - **Envoyer Résultat** : Ce bouton permet de renvoyer le résultat calculé (la valeur convertie) à **CurrencyActivity** et de fermer **ConversionActivity**.

- **Terminer** : Ce bouton ferme simplement **ConversionActivity** sans renvoyer de résultat mais renvoie un signal d'annulation à **CurrencyActivity**. Un **Toast** sera affiché pour informer l'utilisateur qu'aucun résultat n'a été renvoyé.

2. **Préparer CurrencyActivity pour recevoir un résultat**

- Avant de lancer ConversionActivity, il faut enregistrer CurrencyActivity pour pouvoir recevoir un résultat.
- Cela se fait grâce à la fonction **registerForActivityResult()** :

```
private val conversionLauncher =
 registerForActivityResult (ActivityResultContracts.StartActivityForResult()) { result ->
        if (result.resultCode == RESULT_OK) {
            // Récupération du résultat envoyé par ConversionActivity
            val convertedValue = result.data?.getStringExtra("RESULT_VALUE")
            binding.txtResult.text = convertedValue
        } else if (result.resultCode == RESULT_CANCELED) {
            Toast.makeText(this, "Aucun résultat reçu", Toast.LENGTH_SHORT).show()
        }
    }
```

Explication :

- registerForActivityResult() permet de gérer le retour d'une autre activité de manière moderne (remplace startActivityForResult). Cette fonction joue le rôle de la "callback" qui sera exécutée automatiquement quand l'activité ConversionActivity se termine.
- result.resultCode indique si l'opération a été réussie ou annulée :
  RESULT_OK = succès → un résultat a été renvoyé.
  RESULT_CANCELED = échec → aucune donnée renvoyée.
- result.data contient l'intent envoyé par l'activité fille, d'où l'on récupère le résultat.

3. **Lancer ConversionActivity depuis CurrencyActivity**

   L'appel à ConversionActivity ne se fait plus avec startActivity(), mais avec l'objet **conversionLauncher** défini ci-dessus :

```
val intent = Intent(this, ConversionActivity::class.java)
conversionLauncher.launch(intent)
```

Explication :

- Cela permet de lier le lancement de l'activité avec la réception de son résultat.
- La méthode launch() permet de démarrer l'activité fille en attendant un résultat via le **ActivityResultLauncher** créé.

4. **Envoyer / Retourner le résultat depuis ConversionActivity**

   Dans ConversionActivity, le bouton **Envoyer Résultat** doit :

   1. Créer un Intent vide ;
   2. Y insérer le texte du résultat (putExtra()) ;
   3. Appeler setResult(RESULT_OK, intent) pour signaler que l'opération s'est déroulée avec succès.
   4. Fermer l'activité avec finish().

- **Bouton Envoyer Résultat :**

```
binding.btnSendResult.setOnClickListener {
    val result = binding.tvConvertedValue.text.toString()
    val resultIntent = Intent()
    resultIntent.putExtra("RESULT_VALUE", result)
    setResult(RESULT_OK, resultIntent)
    finish()
}
```

5. **Annuler l'opération depuis ConversionActivity / Retour sans résultat**

- **Bouton Terminer :** renvoie simplement un code d'annulation.

```
binding.btnCancel.setOnClickListener {
    setResult(RESULT_CANCELED)
    finish()
}
```

Explication :

- Ici, on ne renvoie pas de donnée.
- RESULT_CANCELED indique que l'opération a été annulée.
- **CurrencyActivity** affichera un Toast pour informer l'utilisateur.

Récap.

⇨ La communication entre activités se fait via des Intent et les codes RESULT_OK / RESULT_CANCELED.

⇨ L'utilisation de registerForActivityResult() est la méthode moderne pour gérer ces échanges.

⇨ On doit toujours considérer les deux cas : succès ou annulation, pour éviter des comportements inattendus dans l'application.

Partie 4 : Activité "TemperatureActivity"

1. Ajoutez une nouvelle activité « **TemperatureActivity** » avec l'interface correspondant à la figure 1.
2. Modifiez l'activité **WelcomeActivity** :
   - Dans **WelcomeActivity**, ajoutez un bouton nommé **Temperature Conversion**.
   - Renommez le bouton de conversion existant en **Currency Conversion**.
   - Le bouton **Temperature Conversion** doit permettre de naviguer vers **TemperatureActivity**.

3. Fonctionnalités de TemperatureActivity

- La zone de saisie doit accepter des nombres à virgule (float).
- Implémentez les conversions dans les deux sens selon le **bouton radio** sélectionné :
  - Celsius → Fahrenheit : Tf = (9/5) * Tc + 32
  - Fahrenheit → Celsius : Tc = (5/9) * (Tf - 32)
- Si l'utilisateur clique sur un bouton radio alors que le champ de saisie est vide, affichez un **Toast** d'avertissement.
- Si la case à cocher **Arrondir** est sélectionnée, le résultat doit être arrondi avant affichage.

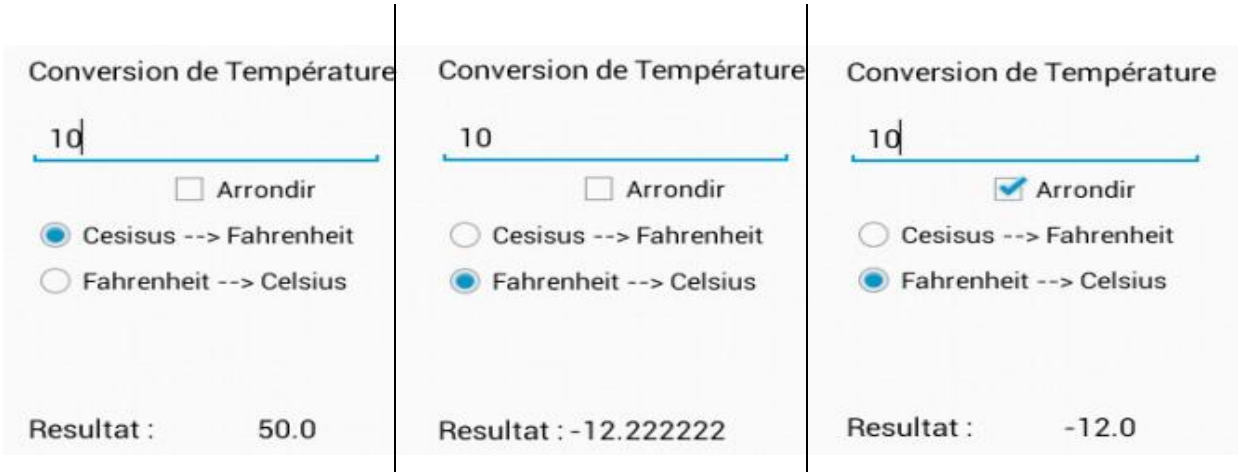


Fig 1. Interface de "TemperatureActivity" et les résultats obtenus selon le choix

BON TRAVAIL